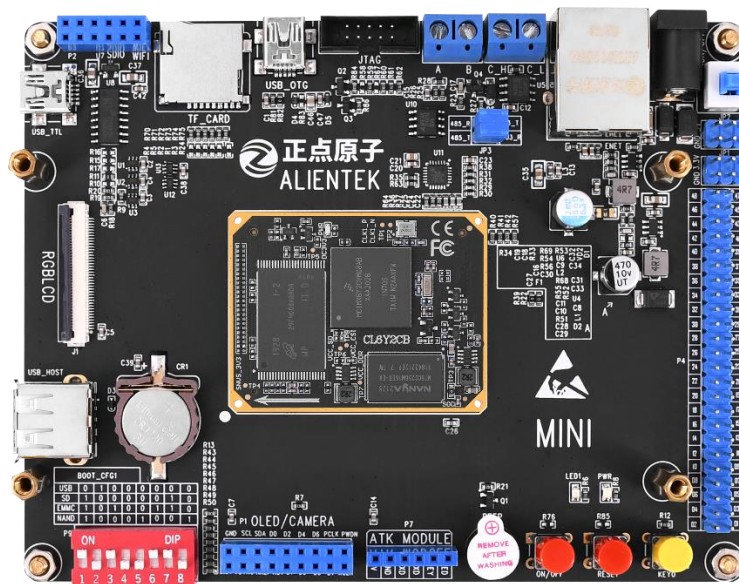
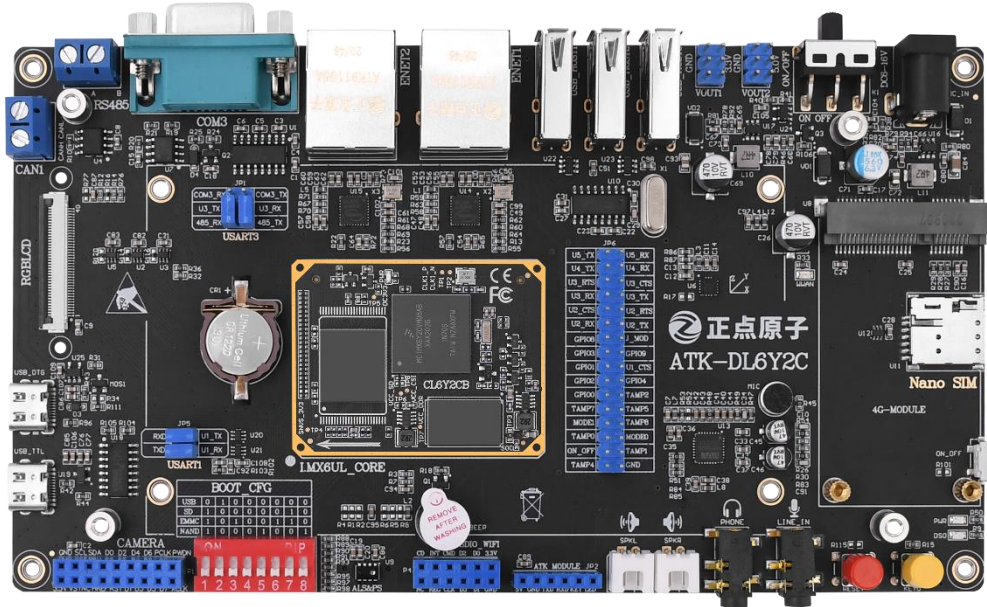


出厂系统 UI 使用说明 V1.0





正点原子公司名称 : 广州市星翼电子科技有限公司

原子哥在线教学平台 : www.yuanzige.com

开源电子网 / 论坛 : <http://www.openedv.com/forum.php>

正点原子淘宝店铺 : <https://openedv.taobao.com>

正点原子官方网站 : www.alientek.com

正点原子 B 站视频 :

<https://space.bilibili.com/394620890>

电话: 020-38271790 传真: 020-36773971

请关注正点原子公众号, 资料发布更新我们会通知。

请下载原子哥 APP, 数千讲视频免费学习, 更快更流畅。



扫码关注正点原子公众号



扫码下载“原子哥”APP

文档更新说明

版本	版本更新说明	负责人	校审	发布日期
V1.0	初稿:	正点原子 linux 团队	正点原子 linux 团队	2024.12.05

目录

前言	5
第一章 Qt UI	7
1.1 Qt UI 简介	8
1.2 Qt UI 源码简介	8
1.3 Qt UI 源码编译	9
1.4 ui 文件夹介绍	10
1.5 用户自定义 App	10
1.6 用户如何添加 App	11
1.7 Qt UI 是如何启动的	12
1.8 移除出厂 UI	12
第二章 App 添加资源文件	13
2.1 music App 添加歌曲	14
2.2 player App 添加视频文件	14
2.3 photoview App 添加图片	14
第三章 APP 说明	15
3.1 主界面	16
3.2 相机 App	16
3.3 位置 App	16
3.4 测试 App	16
3.5 设置 App	17
3.6 更多 App	17
3.7 App 后台管理与内存管理	17

前言

正点原子基于 I.MX6U 开发板开发了一套 Qt UI, 笔者开发的 UI 是演示开发板的功能、测试 UI 能实现的效果、其中也会验证开发板的是否缺失某些库, 可以验证某些 UI 库是否存在 BUG 等等。一套好的 UI 就可以体现系统是否好用!

正点原子开发的 Qt UI, 用户可以参考来做毕业设计(如数码相册, 音乐播放器设计等), 工业控制, 屏幕演示, 性能演示, 广告机, 医疗显示交互, 自动售卖机交互等等用到屏幕的场景。

免责声明

本文档所提及的产品规格和使用说明仅供参考, 如有内容更新, 恕不另行通知; 除非有特殊约定, 本文档仅作为产品指导, 所作陈述均不构成任何形式的担保。本文档版权归广州市星翼电子科技有限公司所有, 未经公司的书面许可, 任何单位和个人不得以营利为目的进行任何方式的传播。

为了得到最新版本的产品信息, 请用户定时访问正点原子资料下载中心或者与淘宝正点原子旗舰店客服联系索取。感谢您的包容与支持。

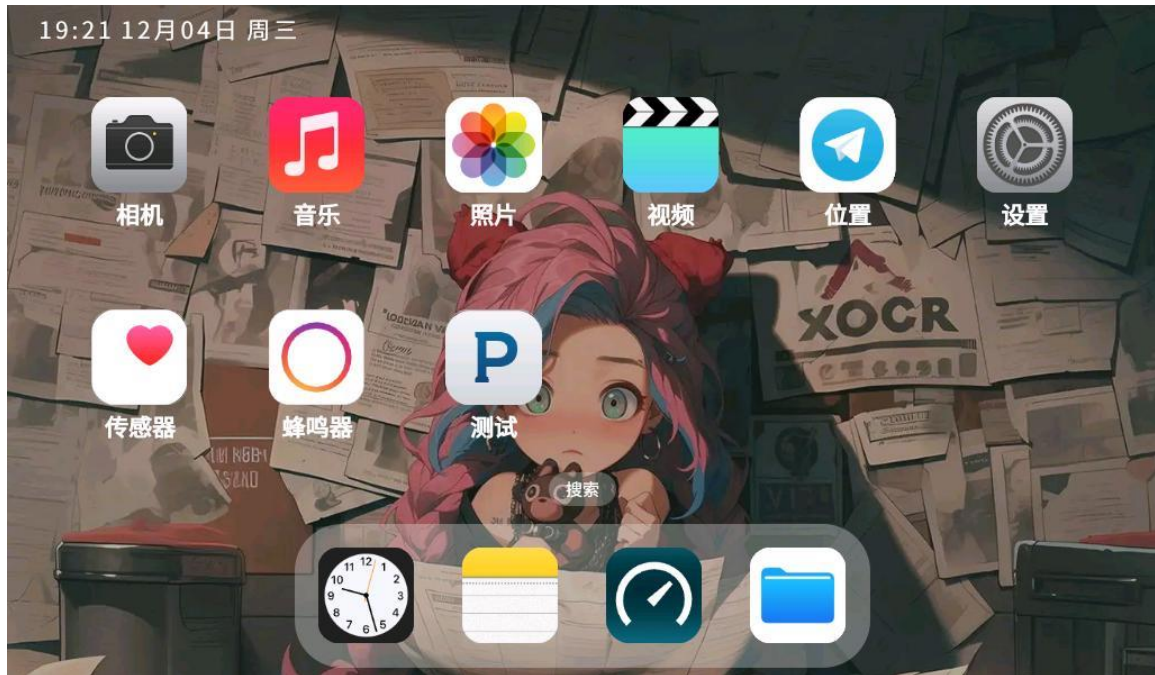
第一章 Qt UI

本章将介绍 Qt UI 相关内容。

1.1 Qt UI 简介

正点原子开发板默认的界面是使用 QML 与 C++ 编程开发的, 由正点原子 Linux 团队开发, 其界面开源。使用的 Qt 版本为 Qt5.12.9, 同时不适用于 Qt6。Qt6 和 Qt5 差别很大, 目前 Qt5 在市场上资料多, 建议大家还是以 Qt5 来学习。

注意由于源码比较庞大, 本文档不作源码解释, 涉及的知识非常多。设计的界面如下。



1.2 Qt UI 源码简介

在开发板光盘网盘-基础资料/01、程序源码/05、Qt 综合例程/systemui.tar.gz。请注意, 早期板子开发的 UI 的名称叫 QDesktop, 现在的 UI 界面名称叫 systemui。因为笔者想让它更系统化。好了现在让我们拷贝这个 systemui.tar.gz 到 Ubuntu 虚拟机上进行编译吧。请注意, 某些 App 必须使用正点原子提供的交叉编译器才可以编译过并且不适配 Windows/Ubuntu, 只针对嵌入式 Qt 设计, 在 Ubuntu 或者 Windows 是无法体验某些功能的。

请参考用户手册下的【正点原子】I.MX6U 开发板文件拷贝及固件更新参考手册, 将 systemui.tar.gz 拷贝到 Ubuntu 下。如下图, 笔者已经将它拷贝到 Ubuntu 下了。

```
alientek@ubuntu:~/qt$ ls
systemui.tar.bz2
alientek@ubuntu:~/qt$
```

将 systemui.tar.bz2 源码解压, 使用下面指令。

```
tar xf systemui.tar.bz2
```

```
alientek@ubuntu:~/qt$ ls
systemui systemui.tar.bz2
alientek@ubuntu:~/qt$ cd systemui/
alientek@ubuntu:~/qt/systemui$ ls
beep      client  fileview location  pcba      Repcs  settings  ui
build.sh  clock   key      music    photoview sensor  systemui
camera    dashboard led      notepad  player    server  template
alientek@ubuntu:~/qt/systemui$
```


1.3 Qt UI 源码编译

请根据 01【正点原子】I.MX6U 用户快速体验 4.2 小节先安装交叉编译器。

直接执行脚本 build.sh all 即可编译全部项目。

```
./build.sh all          # 编译全部项目
./build.sh cleanall     # 如果你需要清除, 执行此指令
```

如果不需要全部编译, 那么下面教大家命令行单独编译一个项目。

进入 systemui 文件夹下, 如下图。Qt 的项目都是 xx.pro, Qt 就是通过 xx.pro 来管理它的项目的, 所以我们编译的项目里必须有 xx.pro。

```
alientek@ubuntu:~/qt/systemui/systemui$ ls
apklistmodel.cpp  BottomApp.qml  launchintent.h  main.qml  Page2.qml  systemui.pro
apklistmodel.h    launchintent.cpp  main.cpp        Page1.qml  qml.qrc    systemui.qrc
alientek@ubuntu:~/qt/systemui/systemui$
```

使能交叉编译器。

```
source /opt/fsl-imx-x11/4.1.15-2.1.0/environment-setup-cortexa7hf-neon-poky-linux-gnueabi
```

用 qmake 生成 Makefile 用于编译 Qt 项目。

```
qmake .          # “.” 表示当前路径
```

```
alientek@ubuntu:~/qt/systemui/systemui$ source /opt/fsl-imx-x11/4.1.15-2.1.0/environment-setup-cortexa7hf-neon-poky-linux-gnueabi
alientek@ubuntu:~/qt/systemui/systemui$ qmake .
Info: creating stash file /home/alientek/qt/systemui/systemui/.qmake.stash
alientek@ubuntu:~/qt/systemui/systemui$ ls
apklistmodel.cpp  launchintent.cpp  main.qml  Page2.qml  systemui.qrc
apklistmodel.h    launchintent.h    Makefile  qml.qrc
BottomApp.qml     main.cpp          Page1.qml  systemui.pro
alientek@ubuntu:~/qt/systemui/systemui$
```

开始编译, 我们直接输入 make 或者 make -j 16, j 后面的数字加快编译速度。

```
make
```

```
alientek@ubuntu:~/qt/systemui/systemui$ make
arm-poky-linux-gnueabi-g++ -march=armv7ve -mfpu=neon -mfloat-abi=hard -mcpu=cortex-a7 -s
ysroot=/opt/fsl-imx-x11/4.1.15-2.1.0/sysroots/cortexa7hf-neon-poky-linux-gnueabi -c -pipe -
-sysroot=/opt/fsl-imx-x11/4.1.15-2.1.0/sysroots/cortexa7hf-neon-poky-linux-gnueabi -O2 -DLI
NIX=1 -DLINUX=1 -DLINUX=1 -DLINUX=1 -DLINUX=1 -DLINUX=1 -std=gnu++11 -Wall -W -D_REENTRANT
-fPIC -DQT_NO_DEBUG -DQT_QUICK_LIB -DQT_GUI_LIB -DQT_QML_LIB -DQT_REMOTEOBJECTS_LIB -DQT_NE
TWORK_LIB -DQT_CORE_LIB -I. -I../server -I/opt/fsl-imx-x11/4.1.15-2.1.0/sysroots/cortexa7hf
-neon-poky-linux-gnueabi/usr/include/QtQuick -I/opt/fsl-imx-x11/4.1.15-2.1.0/sysroots/corte
xa7hf-neon-poky-linux-gnueabi/usr/include/QtGui -I/opt/fsl-imx-x11/4.1.15-2.1.0/sysroots/co
rtexa7hf-neon-poky-linux-gnueabi/usr/include/QtQml -I/opt/fsl-imx-x11/4.1.15-2.1.0/sysroots
/cortexa7hf-neon-poky-linux-gnueabi/usr/include/QtRemoteObjects -I/opt/fsl-imx-x11/4.1.15-2
.1.0/sysroots/cortexa7hf-neon-poky-linux-gnueabi/usr/include/QtNetwork -I/opt/fsl-imx-x11/4
.1.15-2.1.0/sysroots/cortexa7hf-neon-poky-linux-gnueabi/usr/include/QtCore -I. -I../Reps -
```

编译成功会生成 systemui 可执行文件。如下图。(请注意, 不要使用除了出厂系统之外的编译器来编译此项目, 正点原子的出厂系统 Qt 库是很完善的了, 自己移植的 Qt 库可能只是部分, 不完整, 编译时可能会缺少部分库。)

```
alientek@ubuntu:~/qt/systemui/systemui$ ls
apklistmodel.cpp  moc_apklistmodel.cpp  qml.qrc
apklistmodel.h    moc_apklistmodel.o    qrc_common.cpp
apklistmodel.o    moc_launchintent.cpp  qrc_common.o
BottomApp.qml     moc_launchintent.o    qrc_qml.cpp
launchintent.cpp  moc_predefs.h         qrc_qml.o
launchintent.h    moc_rep_systemuicommonapi_source.cpp  rep_systemuicommonapi_source.h
launchintent.o    moc_rep_systemuicommonapi_source.o
main.cpp          moc_systemuicommonapiserver.cpp
main.o            moc_systemuicommonapiserver.o
main.qml          Page1.qml
Makefile          Page2.qml
alientek@ubuntu:~/qt/systemui/systemui$
```

当然你可以按【正点原子】I.MX6U 出厂系统 Qt 交叉编译环境搭建文档，使用搭建的 Kit（编译套件）在 QtCreator 上编译，也是可以的。

接下来就是拷贝这个可执行文件到开发板的根文件系统上，参考【正点原子】I.MX6U 开发板文件拷贝及固件更新参考手册的第一章，笔者已经将往开发板发送文件的文档写的很详细了。这些都是很基础的知识，用户需要自己操作。

发送 systemui 可执行文件，替换开发板根文件系统路径下的/opt/ui/systemui 即可。这个就是系统的主界面。

```
root@ATK-IMX6U:~# ls /opt/ui/systemui
/opt/ui/systemui
root@ATK-IMX6U:~#
```

同理，其他 App 也是进入到相应的目录下去编译。然后 App 就是替换到下面的这个路径。（子 App 都是放/opt/ui/src/apps 路径下！）

```
root@ATK-IMX6U:/opt/ui/src/apps# ls
beep camera clock dashboard fileview key led location music notepad pcba photoview player resource sensor settings
root@ATK-IMX6U:/opt/ui/src/apps#
```

1.4 ui 文件夹介绍

在根文件系统里/opt 路径下有一个 ui 文件夹，同时 systemui.tar.bz2 解压之后在顶层目录下也有一个 ui 文件夹。没错，它们是一样的，意思就是 UI（用户界面）。

我们在一小节知道 systemui 就是我们的主界面，src 就是 source 的简写，通常用于指定图片、脚本、资源文件等。所以我们的 App 就放到这个下面。

```
root@ATK-IMX6U:/opt/ui# ls
src systemui
root@ATK-IMX6U:/opt/ui#
```

- src 文件夹: App 及资源文件存放路径
 - systemui: 系统 Qt 主界面
- 再进入 src 文件夹，有如下内容。

```
root@ATK-IMX6U:/opt/ui/src# ls
appicons apps ATK-IMX6U ipad
root@ATK-IMX6U:/opt/ui/src#
```

- appicons: App 图标
 - apps: App 应用程序存放路径
 - ATK-IMX6U: 6U 开发板配置的 App，用户可以自定义显示什么 App。
 - ipad: 界面用的背景图片，用户可以自行替换
- 我们再进入 apps 文件夹，有如下内容。

```
root@ATK-IMX6U:/opt/ui/src/apps# ls
beep camera clock dashboard fileview key led location music notepad pcba photoview player resource sensor settings
root@ATK-IMX6U:/opt/ui/src/apps#
```

上面的都是 Qt App，用户可以在上一步的 ATK-IMX6U 里面的 xxx.cfg 配置文件中配置要显示的 App。

其中 resource 就是上面的 Qt App 用的资源文件，里面包括歌曲，歌词，mp4，文本等。

1.5 用户自定义 App

如果用户直接使用我们开发的 UI 界面，是可以自定义显示哪些 App 的。

```
root@ATK-IMX6U:/opt/ui/src/ATK-IMX6U# ls
apk1.cfg apk2.cfg apk3.cfg
root@ATK-IMX6U:/opt/ui/src/ATK-IMX6U#
```

可以看到有三个 cfg 配置文件。systemui 主界面就是读取这三个文件来显示哪些 App 的, 笔者参考 Ubuntu/ Weston 的设计, 就是读取对应的配置文件。其中 apk1.cfg 是页面一中间的 App 显示的 App。apk2.cfg 是页面二显示的 App。apk3.cfg 是页面一底部显示的 App。目前设计两页, 如需要更多 App 页面, 请自行修改 systemui 源码!

查看 apk1.cfg 的内容如下。

```
root@ATK-IMX6U:/opt/ui/src/ATK-IMX6U# cat apk1.cfg
appicons/camera.png 相机 camera
appicons/music.png 音乐 music
appicons/photoview.png 照片 photoview
appicons/player.png 视频 player
appicons/location.png 位置 location
appicons/settings.png 设置 settings
appicons/sensor.png 传感器 sensor
appicons/beep.png 蜂鸣器 beep
appicons/pcba.png 测试 pcba
root@ATK-IMX6U:/opt/ui/src/ATK-IMX6U#
```

必须按如上的格式写需要显示的 App。并且用一个空格隔开, 第一列是 app 的图标, 同时用户要查看 appicons 下面有什么图标, 用户可以自己放置自己的图标。第二列是 App 名称, 第三列是应用程序名字, 也就是 src/apps/下的应用程序名称。用户可以尝试删除最后一行 pcba 的显示, 然后保存文件, 重启之后就不显示 pcba App 了。

1.6 用户如何添加 App

用户拷贝 template 模板开发新的 App。

```
alientek@ubuntu:~/qt/systemui$ ls
beep      client  fileview  location  pcba      Repcs  settings  ui
build.sh  clock   key       music     photoview sensor  systemui
camera    dashboard led       notepad   player    server  template
```

现在我们来拷贝一个模板来开发新的 App。拷贝整个 template 命名为 qttest

```
cp -r template qttest
```

```
alientek@ubuntu:~/qt/systemui$ cp -r template qttest
alientek@ubuntu:~/qt/systemui$ ls
beep      client  fileview  location  pcba      qttest  server  template
build.sh  clock   key       music     photoview Repcs   settings ui
camera    dashboard led       notepad   player    sensor  systemui
```

进入 qttest 文件夹, 将 template.pro 命名为 qttest.pro

```
mv template.pro qttest.pro
```

```
alientek@ubuntu:~/qt/systemui/qttest$ mv template.pro qttest.pro
alientek@ubuntu:~/qt/systemui/qttest$ ls
AppMainBody.qml main.cpp main.qml qml.qrc qttest.pro
alientek@ubuntu:~/qt/systemui/qttest$
```

然后打开 main.qml 文件, 将 main.qml 里的 template 替换为 qttest, 如下图。

```
* @brief      main.qml
* @author     Deng Zhimao
* @email      dengzhimao@alientek.com/1252699831@qq.com
* @date       2024-04-12
* @link       http://www.openedv.com/forum.php
*****/
import QtQuick 2.12
import QtQuick.Window 2.12
import com.alientek.qmlcomponents 1.0
Window {
    visible: true
    width: Screen.desktopAvailableWidth
    height: Screen.desktopAvailableHeight
    id: window
    x: 0
    y: 0
    color: "transparent"
    Client {
        programmerName: "qttest"
    }
}
"main.qml" 23L, 702C                                21,31 底端
```

之后我们就可以按上面的方法编译 App 了。将编译的程序拷贝到 src/apps/下。

然后再找一个 120x120（其他正方形规格的图标都行）的 png 图标，命名为 qttest.png，放于 appicons 下。或者你在 appicons 复制一个图标命名为 qttest.png 也行。

然后按 1.5 小节添加自定义 App，如下图，笔者将 qttest 添加到第一页，显示 App 的名称为“app 测试”。

```
root@ATK-IMX6U:/opt/ui/src/ATK-IMX6U# cat apk1.cfg
appicons/camera.png 相机 camera
appicons/music.png 音乐 music
appicons/photoview.png 照片 photoview
appicons/player.png 视频 player
appicons/location.png 位置 location
appicons/settings.png 设置 settings
appicons/sensor.png 传感器 sensor
appicons/beep.png 蜂鸣器 beep
appicons/pcba.png 测试 pcba
appicons/qttest.png app测试 qttest
root@ATK-IMX6U:/opt/ui/src/ATK-IMX6U#
```

保存后，重重启开发板，我们就可以看到一个名为“app 测试”的 App 在主界面上了。

请注意，不要修改 main.qml 的界面交互内容，用户可以在 AppMainBody.qml 中添加自己的 App 内容。也就是所有显示都需要基于 AppMainBody.qml 上去开发。这样才不会影响 App 与主界面的交互效果，如果你不基于我们的 UI 去开发/学习，那么用户自行写程序即可。

1.7 Qt UI 是如何启动的

在根文件系统路径下有一个/etc/rc.local 文件，开机最后执行的一个文件就是这个文件，如果你想重启系统 UI 界面，关闭系统 UI 界面，请点击界面上的。设置 App》通用》退出。

1.8 移除出厂 UI

删除出厂 UI，请你删除/opt/ui 这个文件夹。同时修改/etc/rc.local 文件。

```
echo 30000 > /proc/sys/vm/min_free_kbytes
source /etc/profile
/opt/ui/systemui >/dev/null 2>&1 &
exit 0
```

在行首加一个“#”注释掉，就不会执行了

第二章 App 添加资源文件

本章我们来为自带的 App 添加资源文件。

2.1 music App 添加歌曲

请使用 mp3tag 软件为一首 mp3 注入一张 jpg 格式的专辑图片（详细教程请自行百度）。music App 开机读取 mp3 然后会在 artist 目录中生成专辑封面图片。如果你不会为 mp3 添加专辑封面，那么你可以直接放一张与歌曲同名后缀为 jpg 图片放到 artist 中，这样 music 在启动后将不会从 mp3 里读取这个专辑图片了，如果你不放这个 jpg 图片，也是可以的。将使用默认的图片！有兴趣自行研究。

```
root@ATK-IMX6U:/opt/ui/src/apps/resource# ls
artist audio images logo lrc media pcba shell systemui.conf txt volume weather
root@ATK-IMX6U:/opt/ui/src/apps/resource#
```

同理 lrc 歌词是提前放进去的，编码格式必须为 UTF-8。歌词与歌曲同名后缀为 lrc，详细请参考里面自带的歌词。

audio 文件夹就是歌曲的存放目录，请你将自己喜欢的 mp3 添加到此文件夹中，重启音乐播放器即可！

2.2 player App 添加视频文件

如下图 coverplan 是视频封面，由程序自动根据视频生成，movies 下就是视频播放器的素材。参考里面的文件放对应的文件进去即可！特别注意，由于 IMX6ULL 没有硬件解码，只有软件解码，视频分辨率和帧率有要求，请参考 movies 里的视频分辨率与编码格式，使用格式工厂转换出你喜欢的视频。不要放太高分辨率和帧率的视频，否则会卡成 PPT！

```
root@ATK-IMX6U:/opt/ui/src/apps/resource/media# ls
coverplan movies
root@ATK-IMX6U:/opt/ui/src/apps/resource/media#
```

2.3 photoview App 添加图片

相机拍照的文件会存放于/opt/ui/src/apps/resource/images 文件夹下，同时如果你需要显示你的图片请放于此文件夹下。（同理，由于 IMX6ULL 性能限制，图片分辨率不要太高）。

```
root@ATK-IMX6U:/opt/ui/src/apps/resource/images# ls
夏天水果饮料清凉4k壁纸.jpg 小女孩.jpg 小女孩看报.jpg 山河.jpg 房间.jpg 海边.jpg 猫耳可爱女孩子.jpg 石头.jpg 花.jpg 鸟.jpg
root@ATK-IMX6U:/opt/ui/src/apps/resource/images#
```

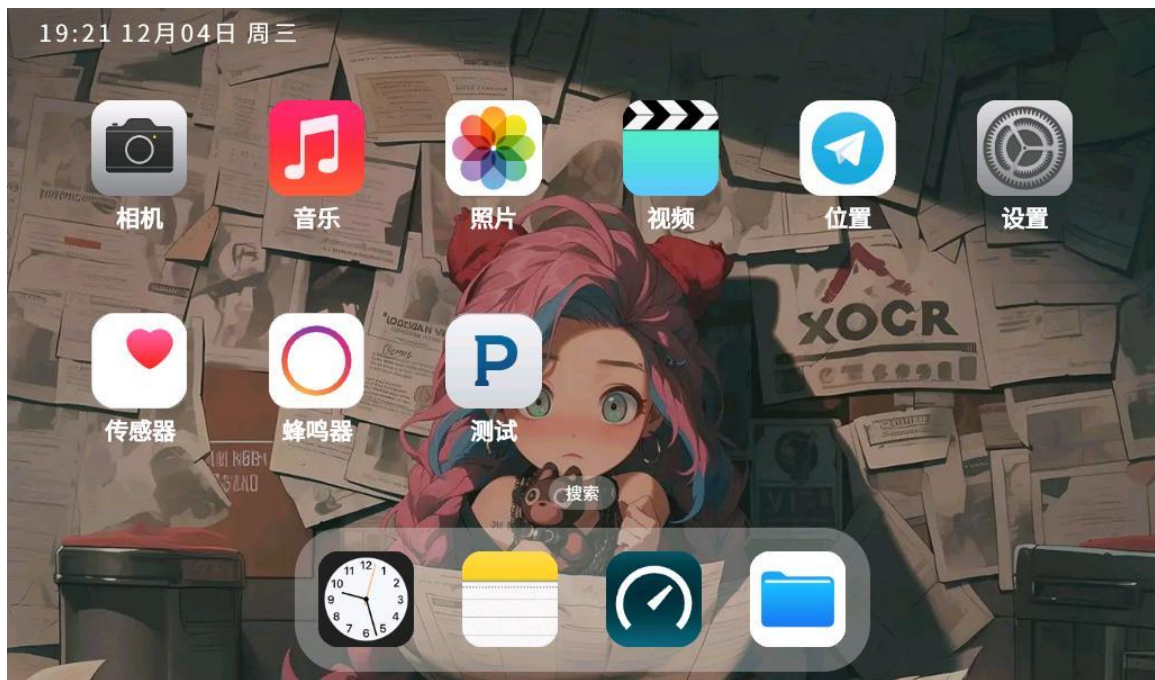
第三章 APP 说明

本章说明使用 APP 注意事项。

3.1 主界面

主界面如下, 由于 Qt 使用的是 Linuxfb 显示插件。容易产生画面撕裂。同时 Linuxfb 是基于 FB 设备显示, 无窗口管理。好比显示屏就是一张白纸, 多个 QtAPP 就是不同的颜料, 若同时开启多个 App, 会出现 **App 抢占屏幕现象**! 而出厂界面很巧妙的实现了不同 APP 切换。也就是当要显示另外一个 App, 当前的 App 先隐藏, 这样来回切换!

对于 IMX6ULL 这种平台, 使用 LinuxFB 显示插件也是很常用的, 也许你见过 XCB/wayland 显示插件显示, 但是由于 6ULL 内存较小, XCB 插件需要 X11 窗口管理系统, wayland 需要 weston 显示服务器。虽然都是轻量级的, 但是对于 6ULL 来说还是需要耗相对多的资源! 所以 LinuxFB 还是比较适合 IMX6ULL 的。



3.2 相机 App

只适配 OV5640。请注意内存使用, 若 App 里拍照过多, 加载的照片就会很多, 容易将内存用完, 尤其是 NAND FLASH 类型的 IMX6ULL 核心板, 只有 256MB DDR! 注意频繁切换摄像头分辨率可能造成界面卡死。

3.3 位置 App

需要插上网线, 并且网线能上网。若一直获取不了位置, 可能是源码里访问位置的网站发生改变。

3.4 测试 App

这个 App 是正点原子出厂时测试部分硬件所用的 App, 需要接上相应的模块来验证硬件功能。用户无需使用此 App! 若自行点击了 App 出现失败也不需要理会, 这个要由我们专业的人员测试, 出厂时验证正确了才出货的。

3.5 设置 App

主要功能是连接 WIFI 支持 SDIO WIFI 和 USB WIFI。注意 USB WIFI 无法获取信号等级, 可能是驱动兼容性产生的, 笔者推荐使用 SDIO WIFI。

3.6 更多 App

其他 App 无需多介绍, 请源码源码。这里因为源码庞大, 不做任何代码解读。仅供用户参考!

3.7 App 后台管理与内存管理

由于 NAND FLASH 类型核心板只有 256MB DDR, 内存较小, 运行多个 Qt APP 容易将内存耗完。所以笔者设计 NAND FLASH 类型核心板不可将 APP 放于后台运行, 点击返回, App 就关闭! 具体源码可参考 Qt 综合例程源码里的 client/systemuicommonapiclient.cpp。

eMMC 则可以指定多个 App 放后台。修改出厂系统/opt/ui/src/apps/resource/systemui.conf 文件配置 App 放后台。

请修改/opt/ui/src/apps/resource/systemui.conf。

```
application-animation=false
background-task=music,player
~
~
```

如上图, 配置了 music, player 放后台, 若你还需要指定 App 放后台, 那么用英文逗号隔开, 继续在后面添加就可以了, 例如添加 camera, 那么改为 background-task=music,player,camera。若全部放后台。background-task=all (推荐使用 EMMC 核心板, 内存有 512MB)。若全部不放后台。background-task=不写任何参数即可!

DDR 内存小的机器建议用户设计 UI 时以单进程为主。保证系统流畅运行系统需要预留一定的内存。在设计 App 时, 尤其是加载图片数据时, 建议使用小分辨率图片, 图标等。过大的图片会占用过多的内存!